COMUNICAÇÃO E SINCRONIZAÇÃO EM SISTEMAS CONCORRENTES

FPPD — Unidade 3

23 de Março, 2026

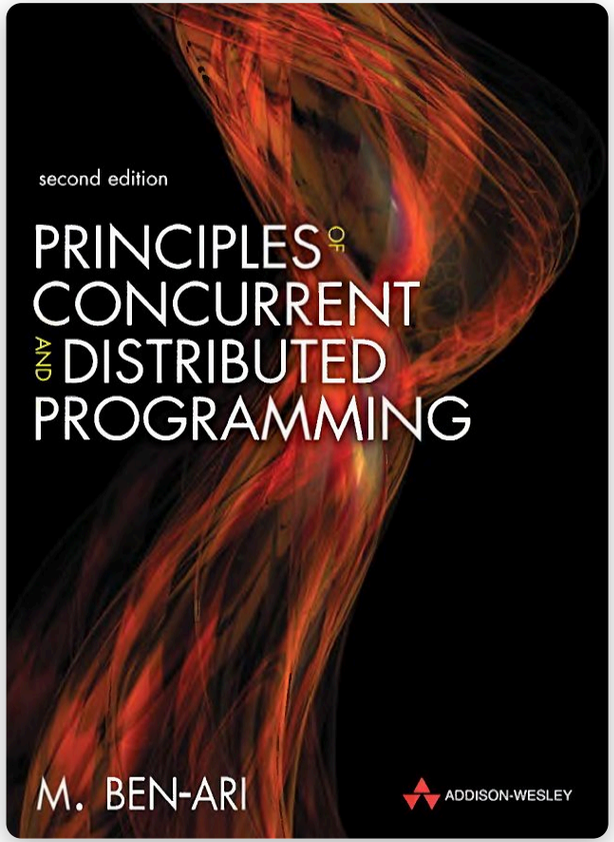
ROTEIRO

1 **Tipos de Comunicação**
Memória compartilhada vs. troca de mensagens

2 **Tipos de Sincronização**
Competição e cooperação

3 **Problemas de Competição**
Seção crítica e jantar dos filósofos

4 **Problemas de Cooperação**
I/O, condição de parada e produtor-consumidor



TIPOS DE COMUNICAÇÃO

Como processos trocam informações

OS DESAFIOS DA CONCORRÊNCIA

Processos que colaboram na resolução de um problema precisam resolver dois desafios fundamentais.

COMUNICAÇÃO

- Trocar **dados** entre processos
- Compartilhar resultados parciais
- Informar sobre eventos

SINCRONIZAÇÃO

- Coordenar a **ordem** de execução
- Esperar que outro processo termine
- Organizar a disputa por recursos

Implementar comunicação e sincronização de forma **eficiente e segura** é muito difícil — quando um programa "congela", normalmente é um erro nessas áreas.

DOIS PARADIGMAS DE COMUNICAÇÃO

MEMÓRIA COMPARTILHADA

- Mesmo **espaço de endereçamento**
- Acesso por **referência**: variáveis globais, ponteiros, índices
- **Sem cópia** de dados — mais eficiente
- Requer controle de **seção crítica**
- Típica em **multiprocessadores**

TROCA DE MENSAGENS

- Espaços de endereçamento **distintos**
- Conteúdo é **copiado** entre processos
- Precisa de **mecanismo**: canais, sockets, MPI
- Facilita a **modelagem** em linguagens modernas
- Típica em **multicomputadores**

COMO ESCOLHER?

A escolha depende do **hardware** — mas em máquinas com memória compartilhada, **ambos são possíveis**.

	Memória Compartilhada	Troca de Mensagens
Requisito	Memória compartilhada (threads)	Qualquer hardware
Risco	**Data races** → seções críticas	**Bloqueio indefinido** → timeouts

"Don't communicate by sharing memory; share memory by communicating."

— Provérbio de Go

COMUNICAÇÃO TAMBÉM SINCRONIZA

Comunicação **síncrona** não apenas transfere dados — ela também **sincroniza** os processos envolvidos.

- Quando o receptor não está pronto, o emissor é **bloqueado**
- Quando o emissor não está pronto, o receptor é **bloqueado**

Um único mecanismo pode resolver **comunicação e sincronização** ao mesmo tempo — esta é a base dos **canais** que veremos nas próximas aulas.

TIPOS DE SINCRONIZAÇÃO

Competição e cooperação

DOIS TIPOS DE SINCRONIZAÇÃO

Processos que colaboram precisam se coordenar. A classificação depende do problema.

COMPETIÇÃO

- Dois ou mais processos **disputam** dados ou recursos compartilhados
- Sem coordenação → **resultado incorreto**
- Exemplos: seção crítica, jantar dos filósofos

COOPERAÇÃO

- Dois ou mais processos precisam se **coordenar** para colaborar
- Sem coordenação → **travamento ou ineficiência**
- Exemplos: I/O, condição de parada, produtor-consumidor

Alguns problemas combinam **competição e cooperação** — como o produtor-consumidor com múltiplos participantes.

SINCRONIZAÇÃO DE COMPETIÇÃO

Quando processos disputam recursos

O PROBLEMA DA SEÇÃO CRÍTICA

Relembrando: quando dois processos acessam dados compartilhados sem proteção, o resultado depende da ordem de execução — uma **data race**.

O PROBLEMA

- Operação `contador++` não é atômica
- Envolve: **ler** → **incrementar** → **escrever**
- Entrelaçamento pode **perder atualizações**

A NECESSIDADE

- **Seção crítica:** trecho que acessa recurso compartilhado
- **Exclusão mútua:** apenas um processo por vez
- Execução **atômica** da seção crítica

Já vimos data race nas aulas anteriores. A pergunta agora é: **como garantir exclusão mútua?** Veremos os mecanismos nas próximas aulas.

SEÇÃO CRÍTICA — BEN-ARI

Algorithm 3.1: Critical section problem	
global variables	
p	**q**

local variables
loop forever
  non-critical section
  preprotocol
  critical section
  postprotocol
	local variables
	loop forever
	  non-critical section
	  preprotocol
	  critical section
	  postprotocol

Fonte: Ben-Ari, *Principles of Concurrent and Distributed Programming*

JANTAR DOS FILÓSOFOS

Problema proposto por Dijkstra (1965)

- **5 filósofos** sentados em uma mesa redonda
- **5 garfos** — um entre cada par de filósofos
- Para comer, cada filósofo precisa de **2 garfos** (esquerdo e direito)
- Cada filósofo alterna entre **pensar** e **comer**
- Cada filósofo é um **processo**; cada garfo é um **recurso compartilhado**

JANTAR DOS FILÓSOFOS — BEN-ARI

Algorithm 6.9: Dining philosophers (outline)

```
    loop forever
p1:     think
p2:     preprotocol
p3:     eat
p4:     postprotocol
```

Fonte: Ben-Ari, *Principles of Concurrent and Distributed Programming*

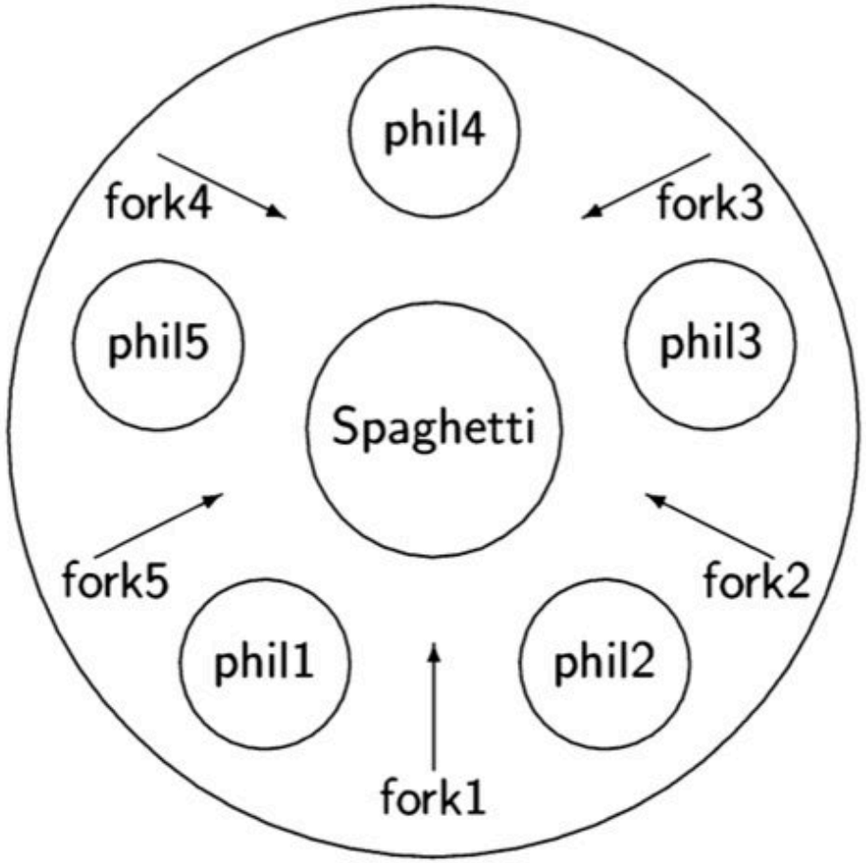
JANTAR DOS FILÓSOFOS — O DESAFIO

É um **problema de seção crítica**: pensar é seção não-crítica; comer (pegar dois garfos) é seção crítica.

- **Sem deadlock** — filósofos não podem ficar travados mutuamente
- **Sem starvation** — todo filósofo deve eventualmente comer
- **Boa concorrência** — múltiplos filósofos devem poder comer ao mesmo tempo

Se todos pegarem o garfo da esquerda ao mesmo tempo → DEADLOCK!
Cada filósofo segura um garfo e espera pelo outro, que está com o vizinho — ninguém come.

JANTAR DOS FILÓSOFOS — DIAGRAMA



Fonte: Ben-Ari, *Principles of Concurrent and Distributed Programming*

SINCRONIZAÇÃO DE COOPERAÇÃO

Quando processos colaboram em um problema

SINCRONIZAÇÃO DE COOPERAÇÃO

Necessária quando dois ou mais processos precisam se **coordenar para colaborar** na resolução de um problema.

- Sem coordenação, o programa pode **não funcionar corretamente** ou **travar**
- Diferente da competição: aqui os processos **precisam um do outro**

OPERAÇÃO DE I/O	CONDIÇÃO DE PARADA	PRODUTOR-CONSUMIDOR
Esperar pela conclusão	Término gracioso	Coordenar produção e consumo

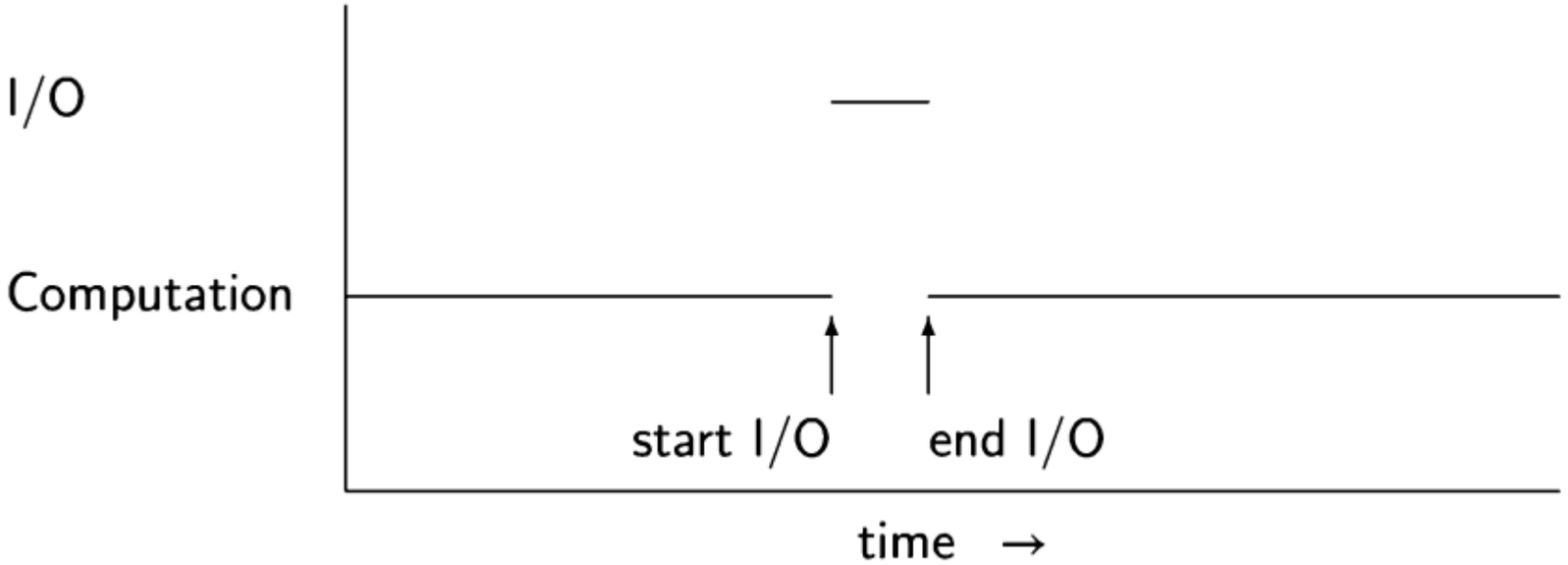
OPERAÇÃO DE ENTRADA E SAÍDA (I/O)

> Cooperação entre o processo que solicita I/O e o processo de atendimento do sistema operacional.

Processo fica **bloqueado** durante a operação de I/O

- O processo que pediu I/O **bloqueia** até que a operação seja concluída
- O SO realiza a operação e **desbloqueia** o processo quando o resultado está disponível
- Exemplo: ler uma string do teclado — o processo espera até a tecla ser pressionada

OPERAÇÃO DE I/O — BEN-ARI



Fonte: Ben-Ari, *Principles of Concurrent and Distributed Programming*

CONDIÇÃO DE PARADA

Coordenação para que o programa termine **graciosamente** — sem interromper processos antes de terminarem e sem deixar processos "pendurados".

O PROBLEMA

- O processo "pai" pode **terminar antes** dos "filhos"
- Trabalho pode ser **perdido silenciosamente**
- Processos podem ficar **pendurados**

A NECESSIDADE

- Esperar que **todos os processos filhos** terminem
- Garantir que nenhum processo é **interrompido prematuramente**
- Sinalizar o **término correto**

Em Go, quando `main()` termina, todas as goroutines são interrompidas — já vimos isso na prática com `sync.WaitGroup`.

PRODUTOR-CONSUMIDOR

Modelo proposto por Dijkstra (1965)

- **Desacopla** produção e consumo — aumenta a concorrência
- Produtores e consumidores executam de forma **concorrente**
- Usado em inúmeras situações reais: recebimento de dados pela rede, tratamento de I/O, filas de tarefas (task queues)

O PROBLEMA DO BUFFER LIMITADO

Bounded Buffer Problem

Na prática, o buffer tem um tamanho **finito** (N posições) — três condições devem ser tratadas.

BUFFER CHEIO

Produtor deve **bloquear** até que haja espaço

🚫 → [▪][▪][▪][▪]

BUFFER VAZIO

Consumidor deve **bloquear** até que haja item

[ ][ ][ ][ ] → 🚫

ACESSO AO BUFFER

Leitura e escrita simultâneas são uma **seção crítica**

data race!

O produtor-consumidor combina **cooperação** (coordenar produção e consumo) com **competição** (proteger acesso ao buffer).

MÚLTIPLOS PRODUTORES E CONSUMIDORES

Múltiplos participantes aumentam a concorrência, mas introduzem **competição adicional**.

RISCOS ADICIONAIS

- Produtores **não podem escrever** na mesma posição
- Consumidores **não podem ler** da mesma posição
- Data races na escrita e leitura

BENEFÍCIOS

- **Mais concorrência** na produção
- **Mais concorrência** no consumo
- Permite **paralelismo** real

RESUMO: OS PROBLEMAS DA CONCORRÊNCIA

Seção Crítica	Competição	Acesso concorrente desprotegido a dados compartilhados — resultado incorreto (data race)
Jantar dos Filósofos	Competição	Disputa por recursos compartilhados — risco de deadlock e starvation
Operação de I/O	Cooperação	Processo espera pela conclusão de uma operação realizada por outro
Condição de Parada	Cooperação	Garantir que o programa termine sem interromper processos
Produtor-Consumidor	Ambos	Coordenar produção e consumo com buffer finito, protegendo acesso compartilhado

MECANISMOS DE SOLUÇÃO

Nas próximas aulas, estudaremos os mecanismos para resolver esses problemas.

CANAIS / MENSAGENS

- Comunicação **e** sincronização em um só mecanismo
- Abordagem preferida em Go
- Funciona sem memória compartilhada

SEMÁFOROS

- Mecanismo de **baixo nível** com contador atômico
- Criado por Dijkstra (1965)
- Base para muitos outros mecanismos

MONITORES

- Mecanismo **estruturado** de alto nível
- Encapsula dados e sincronização
- Base do `synchronized` em Java

RESUMO

- **Comunicação:** memória compartilhada (referência) vs. troca de mensagens (conteúdo)
- **Competição:** disputa por dados/recursos — seção crítica, jantar dos filósofos
- **Cooperação:** coordenação entre processos — I/O, parada, produtor-consumidor
- Produtor-consumidor combina **ambos os tipos** de sincronização
- Comunicação síncrona resolve **comunicação e sincronização** ao mesmo tempo